

Code:

First, Best and Worst Fit of memory

```
#include <stdio.h>
```

```
int main() {
    int num_blocks, num_processes;
    printf("1BM24CS361\n\n");

    printf("Enter number of memory blocks: ");
    scanf("%d", &num_blocks);

    int blocks[num_blocks];
    printf("Enter sizes of %d memory blocks:\n", num_blocks);
    for (int i = 0; i < num_blocks; i++)
        scanf("%d", &blocks[i]);

    printf("Enter number of processes: ");
    scanf("%d", &num_processes);

    int processes[num_processes];
    printf("Enter sizes of %d processes:\n", num_processes);
    for (int i = 0; i < num_processes; i++)
        scanf("%d", &processes[i]);

    int allocation[num_processes];
    int used[num_blocks]; // tracks which blocks are already allocated

    // ----- First Fit -----
    for (int i = 0; i < num_blocks; i++) used[i] = 0;
    for (int i = 0; i < num_processes; i++) allocation[i] = -1;

    printf("\n--- First Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");

    for (int i = 0; i < num_processes; i++) {
        for (int j = 0; j < num_blocks; j++) {
            if (!used[j] && blocks[j] >= processes[i]) {
                allocation[i] = j + 1;
                used[j] = 1; // mark block as taken
                break;
            }
        }
    }
}
```

```

    if (allocation[i] != -1)
        printf("%d\t\t%d\t\t%d\n", i + 1, processes[i], allocation[i]);
    else
        printf("%d\t\t%d\t\tNot Allocated\n", i + 1, processes[i]);
}

```

```

// ——— Best Fit ———

```

```

for (int i = 0; i < num_blocks; i++) used[i] = 0;
for (int i = 0; i < num_processes; i++) allocation[i] = -1;

```

```

printf("\n--- Best Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");

```

```

for (int i = 0; i < num_processes; i++) {
    int best = -1;
    for (int j = 0; j < num_blocks; j++) {
        if (!used[j] && blocks[j] >= processes[i]) {
            if (best == -1 || blocks[j] < blocks[best])
                best = j;
        }
    }
    if (best != -1) {
        allocation[i] = best + 1;
        used[best] = 1; // mark block as taken
        printf("%d\t\t%d\t\t%d\n", i + 1, processes[i], allocation[i]);
    } else {
        printf("%d\t\t%d\t\tNot Allocated\n", i + 1, processes[i]);
    }
}

```

```

// ——— Worst Fit ———

```

```

for (int i = 0; i < num_blocks; i++) used[i] = 0;
for (int i = 0; i < num_processes; i++) allocation[i] = -1;

```

```

printf("\n--- Worst Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");

```

```

for (int i = 0; i < num_processes; i++) {
    int worst = -1;
    for (int j = 0; j < num_blocks; j++) {
        if (!used[j] && blocks[j] >= processes[i]) {
            if (worst == -1 || blocks[j] > blocks[worst])

```

```
        worst = j;
    }
}
if (worst != -1) {
    allocation[i] = worst + 1;
    used[worst] = 1; // mark block as taken
    printf("%d\t\t%d\t\t%d\n", i + 1, processes[i], allocation[i]);
} else {
    printf("%d\t\t%d\t\tNot Allocated\n", i + 1, processes[i]);
}
}

return 0;
}
```

OUTPUT:

C:\Users\Admin\Desktop\wor X + v

18M24CS361

Enter number of memory blocks: 5

Enter sizes of 5 memory blocks:

400 700 200 300 600

Enter number of processes: 4

Enter sizes of 4 processes:

212 512 312 526

--- First Fit ---

Process No.	Process Size	Block No.
1	212	1
2	512	2
3	312	5
4	526	Not Allocated

--- Best Fit ---

Process No.	Process Size	Block No.
1	212	4
2	512	5
3	312	1
4	526	2

--- Worst Fit ---

Process No.	Process Size	Block No.
1	212	2
2	512	5
3	312	1
4	526	Not Allocated

Process returned 0 (0x0) execution time : 34.948 s

Press any key to continue.

|